Innovation Centre for Education

YENEPOYA INSTITUTE OF ARTS, SCIENCE , COMMERCE AND MANAGEMENT

HLD on Project : AI-Enhanced EdTech Application

Student Name: MOHEMMED RAZI

Industry Mentor: Mr. Sumit K Shukla

Guided by:

Table of Contents

1. Introduction
    1.1. Scope of the document
    1.2. Intended Audience
    1.3. System overview
2. System Design
    2.1. Application Design
    2.2. Process Flow
    2.3. Information Flow
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue
3. Data Design
    3.1. Data Model
    3.2. Data Access Mechanism
    3.3. Data Retention Policies
    3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
    7.1. Security Aspects
    7.2. Performance Aspects
8. References

---

1. Introduction

1.1. Scope of the document

1.2. Intended Audience

This document is primarily intended for academic evaluators, university project guides, industry mentors, and prospective developers seeking to understand the structural logic behind the application.

1.3. System overview

The AI-Enhanced EdTech Application is an educational SaaS (Software as a Service) platform. It allows users to dynamically calculate their academic Grade Point Averages and utilizes Machine Learning (scikit-learn) to forecast future academic performance based on user behavioral habits (e.g., weekly study hours). The system integrates modern CI/CD DevOps practices via GitHub Actions and Jenkins.

---

2. System Design

2.1. Application Design

The application utilizes a **Monolithic Client-Server Architecture**:

- **Presentation Layer**: A single-page application construct built with HTML5, Vanilla CSS, and JavaScript.

- **Application Layer**: A Python backend server powered by the Flask framework.
- **Computation Layer**: A decoupled mathematical operations unit housing the `scikitlearn` Machine Learning engine.

2.2. Process Flow

1. The user inputs their academic metrics into the UI.
2. JavaScript triggers an asynchronous HTTP POST request to the API without refreshing the page.
3. The Flask server receives the request, delegates the numbers to the `logic/calculator.py` module.
4. The predictor runs `LinearRegression` inference.
5. The JSON response is routed back to the frontend, instantly updating the UI layer.

2.3. Information Flow

Data moves across the application solely via standard **JSON payloads**. No sensitive data is stored on external file systems between the user client and the host server, ensuring a strict, linear request-response lifecycle.

2.4. Components Design

1. **Frontend Module**: Renders the Glassmorphism UI and houses the core validation logic.
2. **REST API Module**: The standard Flask controller routing traffic.
3. **ML Predictor Module**: The isolated class responsible for training and inferring the underlying statistical data sets.

2.5. Key Design Considerations

- **Separation of Concerns**: Mathematical logic is strictly separated from HTTP webrouting functions, ensuring high testability (as executed in the unit tests).
- **Asynchronous UI**: Prioritized user experience by fetching API responses dynamically in the background.

2.6. API Catalogue

- **`POST /api/calculate_gpa`**: Takes an array of raw percentage grades and computes a 4.0 scale GPA.
- **`POST /api/predict`**: Ingests `study_hours` and `previous_gpa` parameters and returns an ML-predicted future final grade.

---

3. Data Design

3.1. Data Model

For the purposes of this pipeline, the machine learning synthetic data is structured as standard memory matrices.

- *Features (X)*: [Study Hours (int), Last GPA (float)]
- *Target (y)*: [Predicted Final Score (int)]

3.2. Data Access Mechanism

Data is accessed and ingested dynamically via Python's standard local memory mappings (`NumPy` Arrays). No external relational database system (RDBMS) is utilized for this application layer.

3.3. Data Retention Policies

The application maintains strict transient-only data tracking. All predictions and calculations are disposed of post-HTTP response.

3.4. Data Migration

N/A - As there is no persistent relational database mapped to the logic structure, database migrators (like Alembic) are not required.

---

4. Interfaces

The primary interface is a direct Web Browser UI, compatible natively with all modern environments (Chrome, Edge, Firefox) due to the strict reliance on standard vanilla CSS and JS.

---

5. State and Session Management

The application is purely ****stateless****. The server does not issue session cookies or JWT tokens, meaning any prediction request is evaluated entirely independently using only the data present in that instantaneous payload.

---

6. Caching

Given the extremely low computational footprint of the pre-warmed `LinearRegression` algorithm, response times fall consistently below 50ms natively. Consequently, caching mechanisms (e.g., Redis) have been abstained from to preserve architecture simplicity.

---

7. Non-Functional Requirements

7.1. Security Aspects

- Base payload injection protection (Server filters non-integer inputs effectively to prevent execution crashes).
- Boundary limit testing embedded strictly into mathematical logic boundaries to halt overflow attacks.

7.2. Performance Aspects

The pipeline is designed to maintain 99% uptime with response requests returning sub200ms latency limits across local machine deployments.

---

8. References

1. Python Scikit-Learn Documentation (https://scikit-learn.org/)
2. Flask Web Framework Documentation (https://flask.palletsprojects.com/)